PROJECT AND TEAM INFORMATION

Project Title

Driver Monitoring System : Drowsiness & Mobile Distraction Detection with Mobile Integration

Student / Team Information

Team Name: *Team #*	*DriveAI*
Team member 1 (Team Lead) *Ayushi Kumari -2520560034* *Ayushikumari0830@gmail.com*	
Team member 2 *Jyotiraditya Bhatt – 2520560114* *jyotiraditya117@gmail.com*	
Team member 3 *Mayank Pandey-2520560109* *mayankp96755@gmail.com*	

Motivation

Problem Statement

Driver tiredness and using a phone while driving are two issues that can make roads less safe. When someone drives for a time or does not get enough sleep they may feel tired have trouble focusing close their eyes often and yawn. Using a phone can also take attention away from the road. These actions can make accidents more likely especially when the driver does not realize their attention has dropped.

One challenge in watching drivers is spotting these conditions using a camera. Changes in light face movements, camera position and parts of the face being hidden can affect how well the camera can detect. Also one sign, like a blink or a yawn does not always mean the driver is tired. Similarly finding a phone in the car does not always mean the driver is using it.

Therefore a Driver Monitoring System should use face features Machine Learning and mobile phone detection to spot behavior. It should also show a status and give a warning when a dangerous situation is found. The system needs testing to see how well it works in different situations before it can be trusted for real use.

Why We Built This Project

We built this project to see how Computer Vision and Machine Learning can help spot dangers while driving. Our goal is to make a camera-based version that can watch for signs of tiredness and phone use without needing tools.

The system uses OpenCV to look at the camera input and MediaPipe Face Mesh to find points on the face. From these points we measure the Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR). These numbers help us check for eye closing and mouth opening including signs of yawning. We are also making a Random Forest Classifier to decide the drivers state using the features we find.

Along with watching for tiredness we plan to add a part that looks for phones. Combining what both parts find will help the system check for driving instead of just looking at one problem.

Now about half of the project is done. The basic video processing, face points detection EAR/MAR calculation and initial Machine Learning setup are completed. Our next steps are to finish phone detection add risk checking make alerts finish the dashboard and test everything.

Connecting AI with Human Decision-Making

Machine Learning can look at camera data. Find patterns that are hard to see all the time.. An automatic decision should not always be seen as the final answer. For example a driver may yawn because they are tired. One yawn alone does not show how alert they are. A quick eye closing might just be a blink and a phone in view may not be in use.

That is why our system is built to look at both the tiredness detection and the phone use detection. These results go to a part that decides the risk and whether to warn the driver.

We also want a Streamlit dashboard where users can see the video, the detected state and any warnings. This will make the results easier to understand. Let the user make sense of what is happening.

The system is meant to help drivers be more aware not to take over their judgment. The results might not always be right so they should be checked carefully. Not seen as a guarantee of safety.

State of the Art / Current Solution

Existing ways to watch drivers include camera systems devices people wear, car sensors and Advanced Driver Assistance Systems (ADAS). Camera methods look at face features like eye opening, blinking, yawning and head position to find tiredness.

One way to check eyes is the Eye Aspect Ratio (EAR) which uses face points to measure eye opening. Mouth measurements, like the Mouth Aspect Ratio (MAR) can help spot when a mouth is open and possible yawning. Machine Learning models can also be trained using data to decide if a driver is tired.

Looking for phone use can be done with object detection that finds a phone in the camera view.. Finding a phone well depends on how visible it is, the light camera angle and the model used.

Our project brings these methods together into one test system. It uses OpenCV, MediaPipe Face Mesh and a Random Forest Classifier for the part of watching for tiredness. Phone detection, risk checking, alerts and the full dashboard are still being built.

Our Solution

To solve this we are making a Driver Monitoring System that checks the camera for signs of tiredness and phone use. Of looking at just one sign the design uses information from two parts.

1. Real-Time Video Processing

We use Python and OpenCV to get and process video from the camera. The video is the input for watching the drivers face. In the part for distraction the area where phones might be.

The video processing part is done as part of the system.

2. Facial Landmark Detection

We use MediaPipe Face Mesh to find points on the drivers face. These points give the positions needed to look at eye and mouth shape and movement.

This part is. Is the start of the tiredness detection.

3.. Mar Feature Extraction

The system calculates two face measurements:

- **Eye Aspect Ratio (EAR):** Helps measure how open the eyes are and spot long eye closure.

- **Mouth Aspect Ratio (MAR):** Helps measure how open the mouth is and spot yawning.

These numbers help look at the drivers actions. They should be checked over time. Consider differences between drivers.

4. Machine Learning-Based Classification

We chose a Random Forest Classifier for classifying the drivers state. The first part of the Machine Learning setup is done. The model will be. Tested with good data and situations.

The aim is to use the features we find to decide the drivers state and help find tiredness.

5. Mobile Distraction Detection

A part for finding phones in the camera view is being made. This part is meant to find phone use and give results that can be combined with the tiredness results.

The phone detection part and how it joins the system are still being worked on.

6. Risk. Alerts

Once both parts are done their results go to the decision and risk-assessment part. The system is supposed to find possible dangerous situations, like tiredness or possible phone use.

An alert, like a sound or light will happen when the set conditions show a risk. Sending phone alerts may also be part of the version.

7. Current Progress

About half of the project is done. The camera input, video processing, MediaPipe face points detection, EAR/MAR calculation and the first Random Forest steps are done.

The work left includes finishing phone detection putting the parts together making the risk assessment and alert parts finishing the Streamlit dashboard and testing everything.

Project Goals and Milestones

Project Goals

The goal of this project is to build a Driver Monitoring System that uses Computer Vision and Machine Learning to find signs of tiredness and phone use. The system uses a camera to watch the driver and gives info about situations.

One goal is to look at face points and calculate EAR and MAR for tiredness. Another is to make a Random Forest classification system that uses the features to decide the drivers state.

We also want to make a part that finds phones and combines both parts results. This will be the base for checking risk and giving a warning.

Finally we plan to make a Streamlit dashboard to show the camera the drivers state, the results and any warnings. The system will be tested in situations to see how good it is and if it can be used as an example.

Project Milestones

Phase 1: System Planning and Technology Selection

We found the problem picked the main tools and made the basic plan for the Driver Monitoring System. The tools chosen are Python, OpenCV, MediaPipe, NumPy, Pandas, Scikit-learn and Streamlit.

Status: Basic planning and tech selection done.

Phase 2: Camera Processing and Facial Landmark Detection

We made the basic camera processing using OpenCV. MediaPipe Face Mesh was used to find face points from video.

Status: Basic camera processing and face point detection done.

Phase 3: Feature Extraction and Machine Learning

We made the EAR/MAR calculation and set up the first Machine Learning system using a Random Forest Classifier. More work and checking is needed to see how good the classification is in situations.

Status: First system done; model checking and improvement ongoing.

Phase 4: Mobile Phone Detection

Next step is to make the part that finds phones, in the camera view. The results will then be made ready to mix with the tiredness results.

Status: In progress.

Phase 5: Risk Assessment and Alert Generation

The results from the drowsiness detection and mobile phone detection modules will be put together to figure out the monitoring situation. The system will then use the set rules to find situations that could be dangerous and create sound or light warnings.

Mobile alerts could be added if they fit into the design.

Status: More work and combining parts still need to be done.

Phase 6: Dashboard, Workflow and Testing

The Streamlit dashboard will be finished to show the video from the camera the drivers state classification, the mobile phone detection results and the warnings. When all the parts are connected the whole system will be tested in light situations different camera angles, different face movements, different eye closing patterns, different yawns and different times when the phone is visible.

Testing will help us see when the system makes mistakes misses things or has problems. Changes will be made based on what we find.

Status: Finishing the dashboard and doing system testing still need to be done.

Expected Outcome

The goal of this project is to create a working example of a Driver Monitoring System that can take a video feed and spot signs of tiredness and using a phone.

The system will use landmarks and measurements of the eyes and mouth to help check for tiredness. A Random Forest Classifier will be used to figure out the drivers state. A different part of the system will be made to find phones. The results from both parts will be used to check for situations that could be risky.

The finished model should give the warnings and a Streamlit dashboard where people can see what is happening. It will also help us see how camera-based Computer Vision and Machine Learning can work together in one system.

At this point the main result is the monitoring process, including camera processing finding facial landmarks taking facial features and the basic system for classification. Detecting phones, checking for risks making warnings adding the dashboard and final testing are still parts of the work.

The project is meant to show a way to monitor drivers that's not too hard to use. How well it works and how reliable it is will need to be checked through testing. The model should not be seen as a replacement for an official safety system in a car.

References

[1] T. Soukupová and J. Čech "Real-Time Eye Blink Detection using Facial Landmarks " *Proceedings of the 21st Computer Vision Winter Workshop* 2016.

Https://cmp.felk.cvut.cz/pub/cmp/articles/cech/Soukupova-CVWW-2016.pdf

[2] Google AI Edge, "Face Landmarker Guide."

https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker

[3] OpenCV, "VideoCapture Class Documentation."

https://docs.opencv.org/5.0/main_modules/classcv_1_1VideoCapture.html

[4] Scikit-learn, "RandomForestClassifier Documentation."

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[5] Streamlit, "Official Documentation."

https://docs.streamlit.io/

[6] Python Software Foundation, "Python Documentation."

https://docs.python.org/3/

[7] NumPy, "NumPy Documentation."

https://numpy.org/doc/

[8] pandas, "pandas Documentation."

https://pandas.pydata.org/docs/

[9] Google AI Edge, "Face Landmarker Guide, for Python."

https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker/python